



XRPL Integration Security Assessment (Phase 2)

Axelar

Version 1.1 – March 11, 2025

1 Executive Summary

Synopsis

During the February of 2025, Axelar engaged NCC Group to conduct a follow-up security assessment of the XRP Ledger's integration with the Axelar Amplifier protocol. This project provides the necessary support for bridging tokens between XRP and connected chains on the Axelar network, and outbound General Message Passing (GMP) from XRPL to other supported chains. One consultant conducted this review in 4 person-days.

The client addressed the reported findings. As such NCC Group has marked them as fixed in the final report.

Scope

NCC Group's evaluation included the following paths from the <https://github.com/commonprefix/axelar-amplifier> repository on commit [ae2d5ba](#):

- XRPL-specific ampd signing and verification off-chain components:
 - <https://github.com/commonprefix/axelar-amplifier/tree/xrpl/ampd/src/xrpl>
 - https://github.com/commonprefix/axelar-amplifier/blob/xrpl/ampd/src/handlers/xrpl_multisig.rs
 - https://github.com/commonprefix/axelar-amplifier/blob/xrpl/ampd/src/handlers/xrpl_verify_msg.rs
- XRPL Multisig Prover: <https://github.com/commonprefix/axelar-amplifier/tree/xrpl/contracts/xrpl-multisig-prover>
- XRPL Gateway: <https://github.com/commonprefix/axelar-amplifier/tree/xrpl/contracts/xrpl-gateway>
- XRPL Voting Verifier: <https://github.com/commonprefix/axelar-amplifier/tree/xrpl/contracts/xrpl-voting-verifier>
- XRPL types: <https://github.com/commonprefix/axelar-amplifier/tree/xrpl/packages/xrpl-types>

This phase of the review was guided by the difference between the [latest commit hash](#) and the [previously reviewed commit hash](#).

Limitations

The NCC Group team focused their review on the scope above, paying attention to common programming pitfalls and security issues. The consultant achieved good coverage on the in-scope changes.

Key Findings

The assessment uncovered 2 findings:

- [Finding "Unreachable Code should be Removed"](#) which highlights an unreachable code path.
- [Finding "Incorrect Token Deserialization"](#) which identifies an incorrect deserialization logic.

Strategic Recommendations

After addressing the findings presented in this review, NCC Group strongly recommends performing a documentation effort throughout the code base to cover the many sparsely documented design decisions.



2 Table of Findings

For each finding, NCC Group uses a composite risk score that takes into account the severity of the risk, application's exposure and user population, technical difficulty of exploitation, and other factors.

Title	Status	ID	Risk
Unreachable Code should be Removed	Fixed	XKE	Low
Incorrect Token Deserialization	Fixed	Q4J	Low



3 Finding Details

Low

Unreachable Code should be Removed

Overall Risk Low
Impact Low
Exploitability Low

Finding ID NCC-E010021-XKE
Component xrpl-multisig-prover
Category Security Improvement Opportunity
Status Fixed

Impact

Unreachable code should be removed to prevent undefined behavior and to avoid confusion as project expands.

Description

The Multisig Prover contract keeps track of the accumulated dust due to the decimal conversion in the cross-chain (non-XRP) token transfers. Previously, the Multisig Prover contract provided an API that the Governance or the Admin accounts could use to claim dust. Presently, this logic has been removed, however a trace of it exists in the following code snippet from the `confirm_prover_message()` function:

```
XRPLUnsignedTx::Payment(_) => {  
    // Do nothing further if TX is not dust claim.  
    if tx_info.original_cc_id.is_some() {  
        return Ok(None);  
    }  
  
    let dust_info = UNSIGNED_TX_HASH_TO_DUST_INFO.load(storage, &unsigned_tx_hash?);  
    DUST.update(  
        storage,  
        &(dust_info.token_id, dust_info.chain),  
        |current_dust| -> Result<DustAmount, ContractError> {  
            match current_dust {  
                Some(current_dust) => current_dust.sub(dust_info.dust_amount),  
                None => panic!("dust amount must be set"),  
            }  
        },  
    );  
    None  
}
```

Previously, the mapping from the unsigned transaction hash and the dust information was saved in the `UNSIGNED_TX_HASH_TO_DUST_INFO` storage as part of the `ExecuteMsg::ClaimDust`'s message handling by the Multisig Prover contract. Since that code has been removed, the `UNSIGNED_TX_HASH_TO_DUST_INFO` storage will be empty and the first highlighted line in the snippet above will return an error. Thereby, the rest of the logic which updates the `DUST` storage will not have an effect. As such, the severity of this finding is set to low.

Recommendation

Remove the unreachable code. Consider returning an error in case `tx_info.original_cc_id` is `None`.

Location

https://github.com/commonprefix/axelar-amplifier/blob/ae2d5ba8cf8130ad9f7d25b9f41338d189f29ce9/contracts/xrpl-multisig-prover/src/xrpl_multisig.rs#L210-L228



Retest Results

2025-03-11 – Fixed

As part of [PR #5](#), commit [2e9869a](#), the unreachable code has been removed.

The linked PR is merged into the `ncc-2nd-audit` branch, as such this finding is considered “Fixed”.



Incorrect Token Deserialization

Overall Risk	Low	Finding ID	NCC-E010021-Q4J
Impact	Undetermined	Component	xrpl-types
Exploitability	Undetermined	Category	Security Improvement Opportunity
		Status	Fixed

Impact

Failure to properly deserialize XRPL tokens from byte-vectors will result in errors.

Description

The `from_vec()` implementation for the `XRPLToken` type depicted below, takes a byte array of size 40, and interprets the first 20 bytes as the issuer's account ID, and attempts to parse the second half of the input array as the currency. However, in the highlighted block, the used range is from `XRPL_ACCOUNT_ID_LENGTH` to `XRPL_CURRENCY_LENGTH`, i.e., `[20, 20]`¹. As a result, the call to initialize the `currency` from an empty array via `try_into()` will result in an `TryFromSliceError` error. This function is not presently used in the codebase.

```
impl KeyDeserialize for XRPLToken {
    type Output = XRPLToken;
    const KEY_ELEMS: u16 = 2;

    fn from_vec(value: Vec<u8>) -> StdResult<Self::Output> {
        if value.len() != 40 {
            return Err(StdError::generic_err("Invalid key length for XRPLToken"));
        }

        let issuer: [u8; XRPL_ACCOUNT_ID_LENGTH] = value[0..XRPL_ACCOUNT_ID_LENGTH]
            .try_into()
            .map_err(|_| StdError::generic_err("Invalid issuer bytes"))?;

        let currency: [u8; XRPL_CURRENCY_LENGTH] = value
            [XRPL_ACCOUNT_ID_LENGTH..XRPL_CURRENCY_LENGTH]
            .try_into()
            .map_err(|_| StdError::generic_err("Invalid currency bytes"))?;

        Ok(XRPLToken {
            issuer: XRPLAccountId(issuer),
            currency: XRPLCurrency(currency),
        })
    }
}
```

Recommendation

Update `value[XRPL_ACCOUNT_ID_LENGTH..XRPL_CURRENCY_LENGTH]` to `value[XRPL_ACCOUNT_ID_LENGTH..(XRPL_ACCOUNT_ID_LENGTH+XRPL_CURRENCY_LENGTH)]`.

Add a unit test to ensure that the `XRPLToken` deserializer works as expected.

1. See Rust's Range documentation for details: <https://doc.rust-lang.org/std/ops/struct.Range.html>.

Location

<https://github.com/commonprefix/axelar-amplifier/blob/ae2d5ba8cf8130ad9f7d25b9f41338d189f29ce9/packages/xrpl-types/src/types.rs#L294>

Retest Results

2025-03-11 – Fixed

As part of [PR #5](#), commit [7a8d4bc](#), the recommended update has been implemented and a unit test is included.

The linked PR is merged into the `ncc-2nd-audit` branch, as such this finding is considered “Fixed”.



4 Finding Field Definitions

The following sections describe the risk rating and category assigned to issues NCC Group identified.

Risk Scale

NCC Group uses a composite risk score that takes into account the severity of the risk, application's exposure and user population, technical difficulty of exploitation, and other factors. The risk rating is NCC Group's recommended prioritization for addressing findings. Every organization has a different risk sensitivity, so to some extent these recommendations are more relative than absolute guidelines.

Overall Risk

Overall risk reflects NCC Group's estimation of the risk that a finding poses to the target system or systems. It takes into account the impact of the finding, the difficulty of exploitation, and any other relevant factors.

Rating	Description
Critical	Implies an immediate, easily accessible threat of total compromise.
High	Implies an immediate threat of system compromise, or an easily accessible threat of large-scale breach.
Medium	A difficult to exploit threat of large-scale breach, or easy compromise of a small portion of the application.
Low	Implies a relatively minor threat to the application.
Informational	No immediate threat to the application. May provide suggestions for application improvement, functional issues with the application, or conditions that could later lead to an exploitable finding.

Impact

Impact reflects the effects that successful exploitation has upon the target system or systems. It takes into account potential losses of confidentiality, integrity and availability, as well as potential reputational losses.

Rating	Description
High	Attackers can read or modify all data in a system, execute arbitrary code on the system, or escalate their privileges to superuser level.
Medium	Attackers can read or modify some unauthorized data on a system, deny access to that system, or gain significant internal technical information.
Low	Attackers can gain small amounts of unauthorized information or slightly degrade system performance. May have a negative public perception of security.

Exploitability

Exploitability reflects the ease with which attackers may exploit a finding. It takes into account the level of access required, availability of exploitation information, requirements relating to social engineering, race conditions, brute forcing, etc, and other impediments to exploitation.

Rating	Description
High	Attackers can unilaterally exploit the finding without special permissions or significant roadblocks.



Rating	Description
Medium	Attackers would need to leverage a third party, gain non-public information, exploit a race condition, already have privileged access, or otherwise overcome moderate hurdles in order to exploit the finding.
Low	Exploitation requires implausible social engineering, a difficult race condition, guessing difficult-to-guess data, or is otherwise unlikely.

Category

NCC Group categorizes findings based on the security area to which those findings belong. This can help organizations identify gaps in secure development, deployment, patching, etc.

Category Name	Description
Access Controls	Related to authorization of users, and assessment of rights.
Auditing and Logging	Related to auditing of actions, or logging of problems.
Authentication	Related to the identification of users.
Configuration	Related to security configurations of servers, devices, or software.
Cryptography	Related to mathematical protections for data.
Data Exposure	Related to unintended exposure of sensitive information.
Data Validation	Related to improper reliance on the structure or values of data.
Denial of Service	Related to causing system failure.
Error Reporting	Related to the reporting of error conditions in a secure fashion.
Patching	Related to keeping software up to date.
Session Management	Related to the identification of authenticated users.
Timing	Related to race conditions, locking, or order of operations.



5 Contact Info

The team from NCC Group has the following primary members:

- Parnian Alimi – Consultant
parnian.alimi@nccgroup.com
- Javed Samuel – Practice Director, Cryptography Services
javed.samuel@nccgroup.com

The team from Axelar has the following primary member:

- Nikolaos Kamarinakis
nikolas@commonprefix.com

